The Minimalist Edge AI Challenge

Pruning, Quantizing, and Distilling Heavy Models for Lean, Green Edge Hardware

Software Optimization Track
One-day team competition 11th ACM/IEEE Symposium on Edge Computing (SEC 2026)
Santa Clara University

1. The Challenge

You will be given a MobileNetV2 already distilled from a heavy ResNet-50 teacher and trained on satellite imagery. Your job is what comes after distillation, and it is the harder half: an already-compact model has no obvious redundancy left, so every parameter you remove and every bit of precision you drop costs accuracy almost immediately.

How much further can you go once the easy wins are taken - and can you prove what it cost you?

There is no single correct pipeline. Which combination of techniques is right depends on tradeoffs you have to discover, measure, and defend.

Eligibility: undergraduate and graduate students at a recognized university, in teams of 2 to 4, one team per participant, any discipline.

Helpful background: Python with PyTorch, some exposure to pruning or quantization, comfort with the command line and virtual environments.

The Green Trifecta

Four goals, scored as one combined objective:

1. **Model Accuracy** - keep predictive performance high under strict constraints.
2. **Model Compression** - shrink parameter count and on-disk footprint.
3. **Resource Efficiency** - minimize peak RAM and inference latency.
4. **Energy Parsimony** - drive down Joules per inference.

These fight each other, harder than usual on a model this size. That conflict is the challenge. A submission that wins on one leg by sacrificing another score below one that finds a defensible balance and explains why.

2. What You Get and What You Submit

Provided at kickoff	Notes
`student_mobilenetv2_distilled.pt`	**Your starting point and baseline.**
`teacher_resnet50_eurosat.pt`	Optional - useful for distillation loss during recovery fine-tuning.
`baseline_metrics.json`	Official baseline numbers. The denominator for every ratio you report.
`val_split.json`	Fixed split. Everyone is scored identically.
EuroSAT, pre-cached	Copy the `./data` folder rather than downloading at the venue.
Starter notebook	Environment check, data loading, measurement scripts.

Resource Requirements

Resource	Minimum (can work)	Good to have
Laptop OS	Windows 10/11, macOS (Intel or Apple Silicon), or Linux	Any recent version
CPU	4 cores	8 cores
RAM	8 GB	16 GB
Free disk space	5 GB	10 GB

Baseline figures

Accuracy	Params	Size (fp32)	MACs	Worst class

`[98.00]%`	`[23.53]` M	`[22.67]` MB	`[4.13]` G	`[XX.XX]%`

MobileNetV2 is already efficient - roughly an order of magnitude smaller than the ResNet-50 it came from. Your headroom is real but narrow, and the scoring bands in Section 9 are calibrated accordingly.

Target device: If possible, all teams profile against the same device on Qualcomm AI Hub. No physical hardware is shipped. If Qualcomm AI Hub configuration is not available, you can use your laptop instead.

Submit one folder containing:

- The compressed model, preferably quantized ONNX. Must accept `(1, 3, 224, 224)`.
- `metrics.json` (Section 8).
- A 1 - 2 page technical report: what you tried, kept, discarded, and why. Include your tradeoff curve.
- All source code, organized so a judge can reproduce your pipeline from the provided checkpoint without contacting you.

Unverifiable claims are scored as if unmade.

3. Setup

Free, open source, cross-platform. Python 3.10+.

Assume CPU only. *No GPUs are provided. Anything needing many epochs will not finish, so if you fine-tune after pruning, keep it short and budget for it deliberately. An unfinished brilliant idea scores below a completed ordinary one.*

Input stays 224×224. *The provided student inherits this from its ResNet-50 teacher. Reducing input resolution is a real optimization, but every submission must accept* `(1, 3, 224, 224)` *so judges can benchmark on one rig.*

macOS - via Homebrew:

```
brew install python@3.11
python3.11 -m venv edge-ai-env && source edge-ai-env/bin/activate
```

Windows - PowerShell:

```
winget install Python.Python.3.11
py -3.11 -m venv edge-ai-env
edge-ai-env\Scripts\Activate.ps1
```

If activation is blocked, run as Administrator once: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, or use `edge-ai-env\Scripts\activate.bat` from Command Prompt.

Linux (Debian/Ubuntu; for Fedora/RHEL use `dnf`):

```
sudo apt update && sudo apt install -y python3.11 python3.11-venv python3-
pip
python3.11 -m venv edge-ai-env && source edge-ai-env/bin/activate
```

Packages - the CPU index URL on Windows/Linux avoids a multi-gigabyte CUDA download:

```
python -m pip install --upgrade pip

# Windows and Linux only:
pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cpu

# macOS: pip install torch torchvision torchaudio
pip install onnx onnxscript onnxruntime thop codecarbon psutil qai-hub
matplotlib
```

`torch`/`torchvision` (BSD) for modeling; `onnx`/`onnxruntime` (MIT/Apache-2.0) for export and benchmarking; `thop` (MIT) for MACs and parameters; `codecarbon` (MIT) for energy; `psutil` (BSD) for RAM; `qai-hub` for device profiling; `matplotlib` for report plots.

Qualcomm AI Hub - create a free account at aihub.qualcomm.com, generate an API token, then `qai-hub configure --api_token YOUR_TOKEN_HERE`. **Do this before the competition starts.**

4. Task and Starting Model

Identify land-use categories - forest, river, residential, crop, and others - from Sentinel-2 satellite imagery, mirroring real environmental monitoring workloads. EuroSAT is 27,000 images across 10 classes, built into torchvision.

```
import json
from pathlib import Path

import torch
from torch import nn
from torch.utils.data import DataLoader, Subset
from torchvision import datasets, models, transforms

transform = transforms.Compose([
    transforms.Resize(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
0.225]),
])
full_dataset = datasets.EuroSAT(root=Path("./data"), download=False,
transform=transform)

# Use the provided split. Your own split will not be comparable, and judges
score on this one.
split_indices = json.loads(Path("val_split.json").read_text())
train_loader = DataLoader(Subset(full_dataset, split_indices["train"]),
batch_size=32, shuffle=True)
val_loader = DataLoader(Subset(full_dataset, split_indices["val"]),
batch_size=32)

student = models.mobilenet_v2(weights=None)
student.classifier[1] = nn.Linear(student.last_channel, 10)
student.load_state_dict(torch.load("student_mobilenetv2_distilled.pt",
map_location="cpu"))
student.eval()
```

The teacher, if you want it. Distillation is already done, so you do not need the ResNet-50. But if you prune and then fine-tune to recover accuracy, training against the teacher's softened outputs often recovers more than hard labels alone. Whether that is worth the CPU time is exactly the kind of question to answer with evidence.

```
teacher = models.resnet50(weights=None)
teacher.fc = nn.Linear(teacher.fc.in_features, 10)
teacher.load_state_dict(torch.load("teacher_resnet50_eurosat.pt",
map_location="cpu"))
teacher.eval()
```

5. Measure First

Reproduce the baseline before optimizing. These numbers are the denominator for every ratio you claim, and reproducing them confirms your environment works.

5.1 Accuracy, overall and per class

Section 9 scores consistency across all 10 classes, so a model that collapses on one scores below one with the same mean spread evenly.

```
def evaluate(model, loader, device):
    """Top-1 accuracy as a fraction."""
    model.eval()
    correct = total = 0
    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            correct += (model(images).argmax(dim=1) == labels).sum().item()
            total += labels.size(0)
    return correct / total


def per_class_accuracy(model, loader, num_classes=10):
    """Confusion matrix and per-class recall."""
    model.eval()
    confusion = torch.zeros(num_classes, num_classes, dtype=torch.long)
    with torch.no_grad():
        for images, labels in loader:
            predictions = model(images).argmax(dim=1)
            for true_label, predicted in zip(labels, predictions):
                confusion[true_label, predicted] += 1
    return confusion, confusion.diag().float() /
confusion.sum(dim=1).clamp(min=1)


device = torch.device("cpu")
student.to(device)
print(f"Accuracy: {evaluate(student, val_loader, device) * 100:.2f}%")
_, recall = per_class_accuracy(student, val_loader)
print(f"Worst class: {min(recall).item() * 100:.2f}%")
```

5.2 Size, parameters, MACs

```
from thop import profile

dummy_input = torch.randn(1, 3, 224, 224)
macs, params = profile(student, inputs=(dummy_input,))
size_mb = Path("student_mobilenetv2_distilled.pt").stat().st_size / (1024 *
1024)
print(f"MACs: {macs / 1e9:.2f} G | Params: {params / 1e6:.2f} M | Size:
{size_mb:.2f} MB")
```

5.3 Latency and peak RAM

Reading `rss` before and after gives memory at two instants, not the peak in between. Sample continuously instead.

```
import os
import threading
import time

import psutil

process = psutil.Process(os.getpid())
peak_rss_mb = process.memory_info().rss / (1024 * 1024)
sampling = True


def sample_rss(interval_s=0.01):
    """Poll resident memory and record the maximum seen."""
    global peak_rss_mb
    while sampling:
        peak_rss_mb = max(peak_rss_mb, process.memory_info().rss / (1024 *
1024))
        time.sleep(interval_s)


sampler = threading.Thread(target=sample_rss, daemon=True)
sampler.start()
start = time.perf_counter()
with torch.no_grad():
    for _ in range(50):
        student(dummy_input)
elapsed = time.perf_counter() - start
```

```
sampling = False
sampler.join()

print(f"Latency: {elapsed / 50 * 1000:.2f} ms | Peak RSS: {peak_rss_mb:.2f}
MB")
```

On Linux and macOS, `resource.getrusage(resource.RUSAGE_SELF).ru_maxrss` gives a lifetime cross-check (kilobytes on Linux, bytes on macOS). Not available on Windows.

5.4 Measure your ONNX model, not just PyTorch

Your submission is an ONNX file, and that is what judges benchmark. Quantization in particular behaves differently in the exported graph. Report ONNX Runtime figures.

```
import numpy as np
import onnxruntime as ort

session = ort.InferenceSession("student_model_int8.onnx",
providers=["CPUExecutionProvider"])
input_name = session.get_inputs()[0].name

dummy = np.random.randn(1, 3, 224, 224).astype(np.float32)
for _ in range(10):  # warm up
    session.run(None, {input_name: dummy})

start = time.perf_counter()
for _ in range(50):
    session.run(None, {input_name: dummy})
print(f"ONNX latency: {(time.perf_counter() - start) / 50 * 1000:.2f} ms")
```

For accuracy, swap `model(images)` for `session.run(None, {input_name: images.numpy()})[0]` in `evaluate`. The RAM sampler and energy block below work the same way - wrap `session.run` instead. Use identical methods for baseline and final model.

5.5 Energy

```
from codecarbon import EmissionsTracker

tracker = EmissionsTracker(measure_power_secs=1, log_level="error")
tracker.start()
with torch.no_grad():
```

```
    for _ in range(100):
        student(dummy_input)
tracker.stop()

joules = (tracker.final_emissions_data.energy_consumed * 3.6e6) / 100
print(f"Energy per inference: {joules:.4f} J")
```

Two things to know. *CodeCarbon silently falls back to a TDP-based estimate when it cannot read power sensors (macOS without* `sudo powermetrics`*, Windows without vendor tooling) - in that mode energy is just latency times a constant. Check its startup log and record the mode in* `metrics.json`*.*

And because these figures come from your laptop rather than the target board, ***energy is scored as a ratio against your own baseline on your own machine****, never as a raw figure against other teams. Measure both on the same machine or the ratio is meaningless.*

6. Compression

Two main techniques, yours to combine, order, and tune. Remember what you are starting from: MobileNetV2 was built for mobile deployment, so the slack that makes compression easy on a ResNet is not there. Expect accuracy to degrade sooner than tutorials suggest.

6.1 Pruning

Remove weights contributing little to the output, via `torch.nn.utils.prune`. Pruning usually needs a short fine-tune afterwards to recover accuracy.

Expect this: *unstructured pruning zeroes weights rather than removing them, so file size, parameter count, and latency will not change at all. That is normal, not a broken setup. Structured pruning, which drops whole channels, is what actually shrinks a model.*

MobileNetV2's depthwise separable convolutions have few parameters per layer, so a ratio that is harmless on a ResNet can be destructive here. Sweep conservatively at first.

Worth investigating: where accuracy falls off a cliff, whether that cliff sits in the same place for every layer, and whether fine-tuning against the teacher's outputs recovers more than hard labels. PyTorch pruning tutorial

6.2 Quantization

Export is required regardless of route, since you submit ONNX:

```
torch.onnx.export(
    student,
    dummy_input,
    "student_model.onnx",
    input_names=["input"],
    output_names=["output"],
    opset_version=13,
    dynamo=False,
)
```

`dynamo=False` keeps you on the TorchScript exporter - recent PyTorch defaults to dynamo, which targets higher opsets and ignores `opset_version=13` without obvious complaint.

Use static (QDQ) quantization, not dynamic. Dynamic needs no calibration data, but emits `ConvInteger` and `DynamicQuantizeLinear` operators that AI Hub's compile targets frequently reject - and on CPU, dynamically quantized convolutions are often *slower* than FP32. Static calibrates on real images and emits the QDQ format device toolchains expect.

Gotcha: *ONNX Runtime's quantizers expect a shape-inferred graph. Quantizing a freshly exported model tends to fail or silently underperform. Run* `onnxruntime.quantization.preprocess` *first.*

Still yours: calibration set size, which layers stay in higher precision, and where quantization sits relative to pruning. Order matters. ORT quantization · PyTorch quantization

6.3 Strategy

Use one technique or both, in any order. Other optimizations are open too - operator fusion, weight clustering, layer removal - provided the input shape stays `(1, 3, 224, 224)` and you document what you did.

We are deliberately not telling you which combination works best. Teams that sweep a parameter, plot the curve, and choose a point with a stated reason score substantially higher than teams applying a default and reporting a single number - even a good one. Record accuracy after every stage; the tradeoff curve is the most valuable artifact in your report.

7. Profiling on Qualcomm AI Hub

All teams should use the same device profile, if provided. Otherwise, they can use their laptops.

```
import qai_hub as hub

device = hub.Device("[PROFILE]")
compile_job = hub.submit_compile_job(
    model="student_model_int8.onnx",
    device=device,
    input_specs={"input": (1, 3, 224, 224)},
)
compile_job.wait()
profile_job =
hub.submit_profile_job(model=compile_job.get_target_model(),
device=device)
profile_job.wait()
print(profile_job.download_profile())
```

If export leaves duplicate `graph.value_info` entries colliding with graph outputs, strip them with `onnx.load`, filter `value_info` against output names, and re-save before submitting.

Submit early. *The free tier queues and every team reaches this step at once. Get a profile for a working intermediate model well before the deadline. If the service is unreachable, substitute your local measurements and say so - no penalty for an outage.*

8. `metrics.json`

Run your final model through the Section 5 scripts, then:

```
report = {
    "team_name": "REPLACE_ME",
    "baseline": {"accuracy": 0.0, "params_million": 0.0, "size_mb": 0.0,
                 "macs_giga": 0.0, "latency_ms": 0.0, "peak_ram_mb": 0.0,
                 "joules_per_inference": 0.0},
    "final": {"accuracy": 0.0, "params_million": 0.0, "size_mb": 0.0,
              "macs_giga": 0.0, "latency_ms": 0.0, "peak_ram_mb": 0.0,
              "joules_per_inference": 0.0},
    "worst_class_accuracy": 0.0,
    "codecarbon_mode": "measured | tdp_estimate",
    "techniques_applied": ["e.g. structured pruning", "static
quantization"],
    "ai_hub_profile_summary": "key figures from your profile job",
    "measurement_machine": "e.g. MacBook Air M2, 16 GB, CPU only",
}
```

```
Path("metrics.json").write_text(json.dumps(report, indent=2))
```

Baseline and final must come from the **same machine** for the ratios to mean anything.

9. Scoring

100 points. Judges re-run every submission on a standardized rig and score from their own measurements. Your `metrics.json` is used to check your claims match reality; large unexplained discrepancies are treated as unreproducible.

Criterion	Points	What is measured
Model Accuracy & Reliability	25	Accuracy retained vs. the provided baseline, plus consistency across all 10 classes
Model Compression	20	Reduction in parameter count and file size
Resource Efficiency	20	Peak RAM and latency, as ratios against your own baseline, cross-checked on AI Hub
Energy Parsimony	20	Joules per inference, as a ratio against your own baseline
Documentation & Reproducibility	10	Can a judge rebuild your pipeline from the provided checkpoint using only your code?
Engineering Judgment	5	Quality of your tradeoff analysis and the reasoning behind your choices

Ratios rather than raw figures on efficiency and energy, because laptops differ and raw numbers would rank hardware rather than engineering. A TDP-estimated CodeCarbon figure is not penalized, but a measured one is stronger evidence.

A documented negative result earns real points under Engineering Judgment. "We tried X, it cost four points of accuracy for two percent size reduction, so we dropped it" is exactly what that criterion rewards.

10. Troubleshooting

- **`pip install torch` slow or timing out** - use the CPU-only index URL.
- **EuroSAT download fails on venue wifi** - use the pre-cached `./data` folder.
- **`qai-hub` authentication error** - re-run `qai-hub configure --api_token` with a fresh token.
- **Out of RAM** - reduce `batch_size` from 32 to 8.
- **PowerShell blocks activation** - use `activate.bat` from Command Prompt.
- **ONNX opset errors** - `opset_version=13` with `dynamo=False`; ensure `onnxscript` is installed.
- **Pruning changed nothing** - expected with unstructured pruning. See Section 6.1.

Links: PyTorch · torchvision · EuroSAT · ONNX · ONNX Runtime · THOP · CodeCarbon · psutil · Qualcomm AI Hub